ADP470S

Applications Development Practice 4

COMPREHENSIVE TERM 1 STUDY NOTES

Cape Peninsula University of Technology | 2026

Data Structures & Algorithms | Java Implementation

TOPICS COVERED:

1. What is an Algorithm? | 2. Big O Notation
3. Arrays (1D, 2D, Multidimensional) | 4. Recursion vs Iteration
5. Factorial & Fibonacci | 6. Linear & Binary Search
7. Bubble Sort | 8. Insertion Sort | 9. Quick Sort | 10. Heap Sort
11. Linked Lists (Singly, Doubly, Circular)

1. What is an Algorithm?

An algorithm is a finite, ordered set of clear, unambiguous instructions that solves a specific problem. Think of it as a recipe — it takes inputs, follows steps, and produces an output.

Five Properties of an Algorithm (must know!)

Property	Meaning	Example
INPUT	Zero or more well-defined inputs	n = 5 passed to factorial
OUTPUT	At least one result produced	Returns 120
DEFINITENESS	Every step is clear and unambiguous	No vague instructions
FINITENESS	Must terminate in a finite number of steps	Loop must end
EFFECTIVENESS	Each step must be basic enough to execute	No impossible operations

Why Analyse Algorithms?

- Different algorithms solve the same problem differently
- We want to know: How fast is it? (Time) How much memory? (Space)
- Big O Notation is the tool we use to measure this

2. Big O Notation — Algorithm Complexity

Big O notation describes how an algorithm's performance SCALES as the input size n grows. It ignores constants and focuses on the dominant growth pattern.

2.1 Time Complexity vs Space Complexity

Type	Measures	Examples
TIME Complexity	How many operations are performed	Loops, comparisons, recursion calls
SPACE Complexity	How much memory is used	Variables, arrays, stack frames

2.2 Common Big O Classes (Slowest to Fastest)

Big O	Name	Example	For n=1000
O(1)	Constant	Array lookup arr[5]	1 operation
O(log n)	Logarithmic	Binary Search	~10 operations
O(n)	Linear	Single loop / Linear Search	1,000 operations
O(n log n)	Linearithmic	QuickSort, HeapSort (avg)	~10,000 operations
$O(n^2)$	Quadratic	Nested loops / Bubble Sort	1,000,000 operations
$O(2^n)$	Exponential	Recursive Fibonacci (naive)	Enormous

2.3 The Golden Rules of Big O

- **Drop constants:** O(2n) → O(n) | $O(5n^2) \rightarrow O(n^2)$
- **Drop lower-order terms:** $O(n^2 + n) \rightarrow O(n^2)$
- **Sequential loops ADD:** O(n) + O(n) = O(2n) → O(n)
- **Nested loops MULTIPLY:** O(n) × O(n) = $O(n^2)$
- **A loop that divides/triples is logarithmic:** j = j*3 → O(log n)

2.4 Big O Code Examples (Exam Favourite!)

Example A — Two Sequential Loops → O(n)

```
int sum = 0;
for (int i = 0; i < n; i++) {  // Loop 1: runs n times → O(n)
    sum = sum + a[i];
}
for (int j = 0; j < n; j++) {  // Loop 2: runs n times → O(n)
    sum = sum + a[j];
}
// Total: O(n) + O(n) = O(2n) → drop constant → O(n)
```

Example B — Nested Loops (tripling inner) → O(n log n)

```
int prod = 1;
for (int i = 1; i < n; i++) {       // Outer: runs n times → O(n)
    for (int j = i; j < n; j=j*3) { // Inner: j triples each step → O(log n)
        prod = prod * j;
    }
}
// Total: O(n) × O(log n) = O(n log n)
```

Example C — Outer steps of 3 + Inner counts down → O(n²)

```
int sum = 0;
for (int i = 0; i <= n-1; i += 3) { // Outer: n/3 iterations
    for (int j = n-1; j > 0; j--) { // Inner: n iterations
        sum = sum + (i * j);
    }
}
// Total: O(n/3) × O(n) = O(n²/3) → drop constant → O(n²)
```

3. Arrays

An array is a collection of elements of the SAME TYPE stored in CONTIGUOUS (adjacent) memory locations. Each element is accessed by its INDEX (starting at 0 in Java).

3.1 One-Dimensional (1D) Array

Think of a 1D array as a row of boxes, each holding one value.
Index: [0] [1] [2] [3] [4]
Values: 10 20 30 40 50

```
// Declaring and initialising a 1D array
int[] numbers = new int[5];          // Creates array of 5 zeros
int[] grades  = {85, 90, 78, 92, 60}; // Direct initialisation

// Accessing elements
System.out.println(grades[0]); // 85  (first element)
System.out.println(grades[4]); // 60  (last element)

// Looping through an array
for (int i = 0; i < grades.length; i++) {
    System.out.println(grades[i]);
}

// Or using enhanced for-loop (for-each)
for (int g : grades) {
    System.out.println(g);
}
```

Operation	Time Complexity	Why
Access element arr[i]	O(1)	Direct index calculation
Search (unsorted)	O(n)	May need to check every element
Insert at end	O(1)	Just place at index
Insert at middle	O(n)	Must shift elements right
Delete from middle	O(n)	Must shift elements left

3.2 Two-Dimensional (2D) Array

A 2D array is an array of arrays — like a TABLE or GRID with rows and columns.

Visualise a 2D array as a grid:
```
       Col 0  Col 1  Col 2
Row 0  [  1,     2,     3  ]
Row 1  [  4,     5,     6  ]
Row 2  [  7,     8,     9  ]
```

```
// Declaring a 2D array (3 rows, 3 columns)
int[][] grid = new int[3][3];

// Direct initialisation
int[][] matrix = {
    {1, 2, 3},   // Row 0
    {4, 5, 6},   // Row 1
    {7, 8, 9}    // Row 2
};

// Access element at row 1, column 2 → value 6
System.out.println(matrix[1][2]); // Output: 6

// Loop through all elements
for (int row = 0; row < matrix.length; row++) {
    for (int col = 0; col < matrix[row].length; col++) {
        System.out.print(matrix[row][col] + "  ");
    }
    System.out.println(); // new line after each row
}
```

3.3 Limitations of Arrays (Why We Need Linked Lists)

- FIXED SIZE — once declared, the size cannot change
- WASTEFUL — if you declare int[100] but only use 10 slots, 90 are wasted
- EXPENSIVE INSERTION/DELETION — shifting elements is O(n)
- SOLUTION: Linked Lists — dynamic, grow/shrink as needed

4. Recursion vs Iteration

Both recursion and iteration are ways to repeat code. Understanding the difference is CRITICAL for your test.

Aspect	RECURSION	ITERATION
How it works	Method calls ITSELF	Uses a LOOP (for/while)
Termination	BASE CASE stops the calls	Loop condition becomes false
Memory (Space)	O(n) — n stack frames	O(1) — just loop variables
Speed (Time)	O(n) — n method calls	O(n) — n iterations
Stack Overflow?	YES — risk for large n	NO — no call stack used
Readability	Elegant, mirrors maths	More verbose but clear
Real World Use?	Natural for tree/graph traversal	Preferred for simple loops

4.1 Stack Frames — The Key Concept

Every time a method calls itself, Java creates a NEW STACK FRAME on the JVM call stack. Each frame stores:

- The return address (where to jump back to)
- The method parameters (value of n)
- Local variables for that call

```
Call Stack for factorial(4):

 TOP → Frame 4: factorial(1) → returns 1   ← BASE CASE
       Frame 3: factorial(2) → 2 × 1 = 2
       Frame 2: factorial(3) → 3 × 2 = 6
 BOT → Frame 1: factorial(4) → 4 × 6 = 24   ← FIRST CALL

 For factorial(n): n frames = O(n) SPACE
 Each frame does O(1) work → n × O(1) = O(n) TIME
```

5. Factorial — Recursive & Iterative

Factorial: n! = n × (n-1) × (n-2) × ... × 2 × 1
Example: 5! = 5 × 4 × 3 × 2 × 1 = 120
Special case: 0! = 1 and 1! = 1 (these are the BASE CASES)

5.1 Recursive Factorial

```
public class Factorial {

    // Recursive method
    public static int factorialRecursive(int n) {
        if (n == 0 || n == 1) { // BASE CASE — stop recursing
            return 1;
        }
        return n * factorialRecursive(n - 1); // RECURSIVE CASE
    }

    public static void main(String[] args) {
        System.out.println(factorialRecursive(5)); // Output: 120
        System.out.println(factorialRecursive(0)); // Output: 1
    }
}
```

Execution Trace — factorial(5)

```
factorial(5) → calls factorial(4)
  factorial(4) → calls factorial(3)
    factorial(3) → calls factorial(2)
      factorial(2) → calls factorial(1)
        factorial(1) = 1  ← BASE CASE (returns 1)
      factorial(2) = 2 × 1 = 2
    factorial(3) = 3 × 2 = 6
  factorial(4) = 4 × 6 = 24
factorial(5) = 5 × 24 = 120  ← FINAL ANSWER
```

5.2 Iterative Factorial

```
public class Factorial {

    // Iterative method
    public static long factorialIterative(int n) {
        long result = 1; // Start with 1 (0! = 1! = 1)
        for (int i = 2; i <= n; i++) {
            result = result * i; // Multiply running total
        }
        return result;
    }

    public static void main(String[] args) {
        System.out.println(factorialIterative(5)); // Output: 120
```

```
        System.out.println(factorialIterative(0)); // Output: 1
    }
}
```

Step-by-Step Trace — factorialIterative(5)

Step	Action	Result
Init	result = 1, loop starts at i = 2	result = 1
i = 2	result = 1 × 2	result = 2
i = 3	result = 2 × 3	result = 6
i = 4	result = 6 × 4	result = 24
i = 5	result = 24 × 5 → loop ends	result = 120 ✓

5.3 Comparison Summary

	Recursive	Iterative
Time Complexity	O(n)	O(n)
Space Complexity	O(n) — n stack frames	O(1) — one variable
Stack Overflow Risk?	YES for large n	NO
Real World Preference?	Avoid for simple problems	PREFERRED

EXAM TIP: ITERATIVE is preferred for factorial because:
1. Uses O(1) space vs O(n) — no stack overflow
2. No method-call overhead
3. Scales to n = 1,000,000 without crashing
Use recursion ONLY when the problem is NATURALLY recursive (trees, graphs).

6. Fibonacci Sequence

Fibonacci: Each number is the sum of the two before it.
Sequence: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...
Formula: fib(n) = fib(n-1) + fib(n-2), base cases: fib(0)=0, fib(1)=1

6.1 Recursive Fibonacci

```
public static int fibRecursive(int n) {
    if (n == 0) return 0;          // BASE CASE
    if (n == 1) return 1;          // BASE CASE
    return fibRecursive(n-1) + fibRecursive(n-2); // TWO recursive calls!
}
```

WARNING: Recursive Fibonacci is VERY SLOW!

fib(5) calls fib(4) and fib(3)
fib(4) calls fib(3) and fib(2) ← fib(3) computed TWICE!
fib(3) calls fib(2) and fib(1) ← repeated over and over

Time Complexity: O(2^n) — exponential! Doubles with each n.
Space Complexity: O(n) — stack depth

6.2 Iterative Fibonacci (Much Better!)

```
public static int fibIterative(int n) {
    if (n == 0) return 0;
    if (n == 1) return 1;
    int prev2 = 0; // fib(0)
    int prev1 = 1; // fib(1)
    int current = 0;
    for (int i = 2; i <= n; i++) {
        current = prev1 + prev2; // sum of previous two
        prev2 = prev1;           // shift window forward
        prev1 = current;
    }
    return current;
}
// Time: O(n)   Space: O(1) — MUCH better than recursive!
```

n	fib(n)	Trace: current = prev1 + prev2
0	0	base case
1	1	base case
2	1	1 + 0 = 1
3	2	1 + 1 = 2
4	3	2 + 1 = 3

5	5	3 + 2 = 5
6	8	5 + 3 = 8

7. Searching Algorithms

7.1 Linear Search

Start at the beginning. Check each element one by one until you find the target or reach the end.

```
public static int linearSearch(int[] arr, int target) {
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == target) {
            return i; // Found! Return index
        }
    }
    return -1; // Not found
}

// Example:
// arr = {10, 25, 8, 42, 15}, target = 42
// Check 10 → No | 25 → No | 8 → No | 42 → YES! returns index 3
```

Case	Complexity	Scenario
Best Case	O(1)	Target is the FIRST element
Average Case	O(n)	Target is somewhere in the middle
Worst Case	O(n)	Target is LAST or not found — must check all n elements

7.2 Binary Search

REQUIRES sorted array. Repeatedly halve the search space by comparing target with the MIDDLE element.

Example: arr = {3, 7, 12, 18, 25, 34, 45, 56, 67, 89}, target = 25

Step 1: low=0, high=9, mid=4 → arr[4]=25 == target → FOUND! Return 4

Example: target = 34
Step 1: low=0, high=9, mid=4 → arr[4]=25 < 34 → search RIGHT half
Step 2: low=5, high=9, mid=7 → arr[7]=56 > 34 → search LEFT half
Step 3: low=5, high=6, mid=5 → arr[5]=34 == target → FOUND! Return 5

```
public static int binarySearch(int[] arr, int target) {
    int low = 0;
    int high = arr.length - 1;

    while (low <= high) {
        int mid = (low + high) / 2; // find middle index
```

```
        if (arr[mid] == target) {
            return mid;              // FOUND
        } else if (arr[mid] < target) {
            low = mid + 1;           // search RIGHT half
        } else {
            high = mid - 1;          // search LEFT half
        }
    }
    return -1; // Not found
}
```

Case	Complexity	Scenario
Best Case	O(1)	Target is the MIDDLE element
Average Case	O(log n)	Each step halves the problem
Worst Case	O(log n)	Target at very end or not found

7.3 Linear vs Binary Search Comparison

	Linear Search	Binary Search
Requires sorted data?	NO	YES — must be sorted
Best Case	O(1)	O(1)
Worst Case	O(n)	O(log n)
Space	O(1)	O(1)
When to use?	Small or unsorted data	Large sorted data

8. Bubble Sort

Bubble Sort compares NEIGHBOURING elements. If the left element is GREATER than the right, SWAP them. After each full pass, the largest unsorted element 'bubbles up' to its correct position at the end.

8.1 Java Code

```
public static void bubbleSort(int[] arr) {
    int n = arr.length;
    for (int i = 0; i < n - 1; i++) {       // outer: n-1 passes
        boolean swapped = false;              // optimisation flag
        for (int j = 0; j < n - 1 - i; j++) { // inner: compare pairs
            if (arr[j] > arr[j + 1]) {
                // SWAP
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = true;
            }
        }
        if (!swapped) break; // already sorted → exit early (Best Case O(n))
    }
}
```

8.2 Step-by-Step Trace — {90, 23, 5, 109, 12}

Step	Comparison	Action	Array State
Pass 1, Step 1	90 vs 23	90 > 23 → SWAP	[23, 90, 5, 109, 12]
Pass 1, Step 2	90 vs 5	90 > 5 → SWAP	[23, 5, 90, 109, 12]
Pass 1, Step 3	90 vs 109	90 < 109 → No swap	[23, 5, 90, 109, 12]
Pass 1, Step 4	109 vs 12	109 > 12 → SWAP → 109 in place ✓	[23, 5, 90, 12, 109]
Pass 2, Step 1	23 vs 5	23 > 5 → SWAP	[5, 23, 90, 12, 109]
Pass 2, Step 2	23 vs 90	23 < 90 → No swap	[5, 23, 90, 12, 109]
Pass 2, Step 3	90 vs 12	90 > 12 → SWAP → 90 in place ✓	[5, 23, 12, 90, 109]
Pass 3, Step 1	5 vs 23	5 < 23 → No swap	[5, 23, 12, 90, 109]
Pass 3, Step 2	23 vs 12	23 > 12 → SWAP → 23 in place ✓	[5, 12, 23, 90, 109]
Pass 4, Step 1	5 vs 12	5 < 12 → No swap → DONE ✓	[5, 12, 23, 90, 109]

8.3 Complexity Analysis

Case	Complexity	Why
Best Case	O(n)	Already sorted — 1 pass, swapped flag exits early
Average Case	O(n²)	Random order — ~n²/2 comparisons
Worst Case	O(n²)	Reverse sorted — max n(n-1)/2 comparisons
Space	O(1)	In-place — only uses temp variable

9. Insertion Sort

Pick the next unsorted element and SLIDE IT LEFT through the sorted portion until it reaches its correct position. Like sorting playing cards in your hand.

9.1 Java Code

```
public static void insertionSort(int[] arr) {
    for (int i = 1; i < arr.length; i++) {
        int key = arr[i];    // Element to insert
        int j = i - 1;
        // Shift elements greater than key one position RIGHT
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;   // Insert key at correct position
    }
}
```

9.2 Step-by-Step Trace — {90, 23, 5, 109, 12}

i	key	Action	Array State
i=1	key=23	90 > 23 → shift 90 right. Insert 23 before 90.	[23, 90, 5, 109, 12]
i=2	key=5	90>5 shift, 23>5 shift. Insert 5 at front.	[5, 23, 90, 109, 12]
i=3	key=109	90 < 109 → no shift. 109 stays.	[5, 23, 90, 109, 12]
i=4	key=12	109>12 shift, 90>12 shift, 23>12 shift, 5<12 stop. Insert 12.	[5, 12, 23, 90, 109] ✓

9.3 Complexity Analysis

Case	Complexity	Why
Best Case	O(n)	Already sorted — inner while never runs
Average Case	O(n²)	Random data — avg n/2 shifts per element
Worst Case	O(n²)	Reverse sorted — maximum shifts every time

Space	O(1)	In-place — only uses key variable

10. Quick Sort

Quick Sort uses DIVIDE AND CONQUER. Choose a PIVOT element, PARTITION the array so all elements less than pivot are on the left and greater are on the right, then recursively sort both sides.

10.1 Steps

1. Choose a PIVOT (often last element, first element, or middle element)
2. PARTITION: rearrange so everything < pivot is left, everything > pivot is right
3. RECURSIVELY sort the left sub-array and right sub-array
4. Base case: sub-array of size 0 or 1 is already sorted

10.2 Java Code

```
public static void quickSort(int[] arr, int low, int high) {
    if (low < high) {
        int pivotIndex = partition(arr, low, high);
        quickSort(arr, low, pivotIndex - 1);  // Sort LEFT half
        quickSort(arr, pivotIndex + 1, high); // Sort RIGHT half
    }
}

public static int partition(int[] arr, int low, int high) {
    int pivot = arr[high]; // last element as pivot
    int i = low - 1;       // index of smaller element
    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            int temp = arr[i]; arr[i] = arr[j]; arr[j] = temp; // swap
        }
    }
    // Place pivot in correct position
    int temp = arr[i+1]; arr[i+1] = arr[high]; arr[high] = temp;
    return i + 1;
}

// Call: quickSort(arr, 0, arr.length - 1);
```

10.3 Trace — {90, 23, 5, 109, 12} (last element as pivot)

Step	Pivot	Action	Result
R1	12 (last)	< 12: {5} | 12 | > 12: {90,23,109}	[5 | 12 | 90,23,109]
R2	—	{5}: size 1 → sorted	[5, 12, ...]
R3	109 (last)	< 109: {90,23} | 109 | > 109: {}	[... | 90,23 | 109]

R4	23 (last)	< 23: {} | 23 | > 23: {90}	[..., 23, 90, 109]
DONE	—	All sub-arrays size 1 → sorted ✓	[5, 12, 23, 90, 109]

10.4 Complexity Analysis

Case	Complexity	Why
Best Case	O(n log n)	Pivot splits array into equal halves every time
Average Case	O(n log n)	Roughly balanced partitions on random data
Worst Case	$O(n^2)$	Pivot is always smallest/largest (already sorted + last pivot)
Space	O(log n)	Recursive call stack depth

TIP: To avoid worst case, use RANDOM PIVOT or MEDIAN-OF-THREE pivot strategy.

11. Heap Sort

Heap Sort uses a data structure called a MAX-HEAP — a binary tree where every parent is GREATER than or equal to its children. Root is always the maximum element.

11.1 Two Phases

- **Phase 1 — Build Max-Heap:** Rearrange the array into a valid max-heap. Starts from the last non-leaf node and heapifies upward.
- **Phase 2 — Extract max repeatedly:** Swap root (max) with the last element, reduce heap size, re-heapify. Repeat n-1 times.

11.2 Trace — {90, 23, 5, 109, 12}

Array as Binary Tree:

```
      90  (index 0)
     /  \
    23    5  (index 1, 2)
   / \
 109  12  (index 3, 4)
```

Phase 1 — Build Max-Heap:

Step	Node (index)	Action	Array State
1	index 1 (val=23)	Children: 109, 12. 109 > 23 → SWAP 23 and 109	[90, 109, 5, 23, 12]
2	index 0 (val=90)	Children: 109, 5. 109 > 90 → SWAP 90 and 109	[109, 90, 5, 23, 12] MAX-HEAP ✓

Phase 2 — Extract max:

Step	Action	Array State
E1	Swap root 109 with last (12). Remove 109. Re-heapify.	[90, 23, 5, 12 | 109]
E2	Swap root 90 with last (12). Remove 90. Re-heapify.	[23, 12, 5 | 90, 109]
E3	Swap root 23 with last (5). Remove 23. Re-heapify.	[12, 5 | 23, 90, 109]
E4	Swap root 12 with last (5). Remove 12.	[5 | 12, 23, 90, 109]

DONE	Only 5 left → SORTED ✓	[5, 12, 23, 90, 109]

11.3 Complexity Analysis

Case	Complexity	Why
Best Case	O(n log n)	Always same — tree operations are always O(log n)
Average Case	O(n log n)	Consistent regardless of input
Worst Case	O(n log n)	GUARANTEED — no bad pivot problem like QuickSort
Space	O(1)	In-place — sorts within the original array
Build Max-Heap	O(n)	Phase 1 is O(n) total (not O(n log n))

12. Sorting Algorithms — Complete Comparison

Algorithm	Best Case	Average Case	Worst Case	Space	Stable?
Bubble Sort	O(n)	O(n²)	O(n²)	O(1)	Yes
Insertion Sort	O(n)	O(n²)	O(n²)	O(1)	Yes
Quick Sort	O(n log n)	O(n log n)	O(n²)	O(log n)	No
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)	No

STABLE sort = equal elements maintain their original relative order.
IN-PLACE sort = uses O(1) extra space (only a few variables for swapping).

HeapSort is the ONLY sort here guaranteed O(n log n) in ALL cases.
QuickSort is usually FASTEST in practice despite O(n²) worst case.

13. Linked Lists

A Linked List is a DYNAMIC data structure made of NODES. Each node stores DATA and a POINTER (reference) to the next node. Unlike arrays, linked lists do NOT store elements in contiguous memory — each node can be anywhere in memory.

13.1 Why Linked Lists? (Array vs Linked List)

Operation	Array	Linked List
Access element by index	O(1) — direct	O(n) — must traverse
Insert at beginning	O(n) — shift all	O(1) — update head pointer
Insert at end	O(1)	O(n) unless tail pointer kept
Delete from middle	O(n) — shift	O(n) to find + O(1) to remove
Size	Fixed at creation	Dynamic — grows/shrinks
Memory	Wastes empty slots	Uses exactly what's needed

13.2 Singly Linked List

Each node has DATA and a NEXT pointer. Last node's next = null.

```
HEAD → [Data|Next] → [Data|Next] → [Data|Next] → null
       Node 1        Node 2        Node 3

Example: HEAD → [10|•] → [20|•] → [30|null]
```

```
// Node class
class Node {
    int data;
    Node next;
    public Node(int data) {
        this.data = data;
        this.next = null;
    }
}

// Singly Linked List
class LinkedList {
    Node head; // Points to first node

    // Insert at beginning — O(1)
    public void insertAtFront(int data) {
        Node newNode = new Node(data);
        newNode.next = head; // New node points to old head
        head = newNode;      // Head now points to new node
    }

    // Insert at end — O(n)
```

```
    public void insertAtEnd(int data) {
        Node newNode = new Node(data);
        if (head == null) { head = newNode; return; }
        Node current = head;
        while (current.next != null) { // Traverse to last node
            current = current.next;
        }
        current.next = newNode; // Last node points to new node
    }

    // Display all elements — O(n)
    public void display() {
        Node current = head;
        while (current != null) {
            System.out.print(current.data + " → ");
            current = current.next;
        }
        System.out.println("null");
    }

    // Delete a node by value — O(n)
    public void delete(int data) {
        if (head == null) return;
        if (head.data == data) { head = head.next; return; }
        Node current = head;
        while (current.next != null && current.next.data != data) {
            current = current.next;
        }
        if (current.next != null) {
            current.next = current.next.next; // Skip the deleted node
        }
    }
}
```

13.3 Limitations of Singly Linked List

- Can only traverse FORWARD (head to tail)
- Cannot go backward to previous node
- To delete a node, need access to its PREVIOUS node → must start from head
- SOLUTION: Doubly Linked List

13.4 Doubly Linked List

Each node has a PREV pointer, DATA, and NEXT pointer. Can traverse in BOTH directions.

```
null ← [prev|Data|next] ↔ [prev|Data|next] ↔ [prev|Data|next] → null
       Node 1              Node 2              Node 3

Example: null ← [null|10|•] ↔ [•|20|•] ↔ [•|30|null]
```

```
class DNode {
    int data;
    DNode prev; // Points to PREVIOUS node
    DNode next; // Points to NEXT node
    public DNode(int data) {
        this.data = data;
```

```
        this.prev = null;
        this.next = null;
    }
}

class DoublyLinkedList {
    DNode head;
    DNode tail; // Optional: keep tail for O(1) end insertion

    public void insertAtEnd(int data) {
        DNode newNode = new DNode(data);
        if (head == null) { head = tail = newNode; return; }
        newNode.prev = tail;  // New node's prev = old tail
        tail.next = newNode;  // Old tail's next = new node
        tail = newNode;       // Update tail
    }

    // Can traverse BACKWARD from tail to head!
    public void displayReverse() {
        DNode current = tail;
        while (current != null) {
            System.out.print(current.data + " ");
            current = current.prev; // Go BACKWARD
        }
    }
}
```

13.5 Circular Linked List

The LAST node's next pointer points BACK to the HEAD (instead of null). Creates a circle — no start or end.

HEAD → [10|•] → [20|•] → [30|•]
↑_____|
(Last node's next points back to HEAD)

Useful for: round-robin scheduling, circular queues, playlists.

```
class CircularLinkedList {
    Node head;

    public void insertAtEnd(int data) {
        Node newNode = new Node(data);
        if (head == null) {
            head = newNode;
            head.next = head; // Points to itself
            return;
        }
        // Traverse to last node (node whose next = head)
        Node current = head;
        while (current.next != head) {
            current = current.next;
        }
        current.next = newNode; // Last node → new node
        newNode.next = head;    // New node → head (circular!)
    }

    public void display() {
```

```
        if (head == null) return;
        Node current = head;
        do {
            System.out.print(current.data + " → ");
            current = current.next;
        } while (current != head); // Stop when we loop back
        System.out.println("(back to head)");
    }
}
```

13.6 Linked List Types Summary

Type	Directions	Last Node Points To	Use Case
Singly	Forward only	null	Simple queue/stack
Doubly	Forward & Backward	null (both ends)	Browser history, undo/redo
Circular Singly	Forward (circles)	HEAD	Round-robin scheduling
Circular Doubly	Both (circles)	HEAD/TAIL	Advanced circular buffers

14. Quick Reference — Complete Complexity Table

Structure / Algorithm	Best	Average	Worst	Space	Notes
Linear Search	O(1)	O(n)	O(n)	O(1)	Works on unsorted
Binary Search	O(1)	O(log n)	O(log n)	O(1)	Needs sorted array
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Stable, in-place
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Stable, in-place
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$	O(log n)	Fastest in practice
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)	Guaranteed O(n log n)
Factorial (Rec)	O(n)	O(n)	O(n)	O(n)	Risk: StackOverflow
Factorial (Iter)	O(n)	O(n)	O(n)	O(1)	Preferred!
Fib (Recursive)	$O(2^n)$	$O(2^n)$	$O(2^n)$	O(n)	Very slow! Avoid
Fib (Iterative)	O(n)	O(n)	O(n)	O(1)	Preferred
Linked List Insert	O(1)	O(n)	O(n)	O(1) per op	Front insert O(1)
Linked List Search	O(1)	O(n)	O(n)	O(1)	No index access

EXAM CHEAT SHEET:

$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n)$
Best Worst

REMEMBER:
• Sequential loops ADD complexities → then drop constants
• Nested loops MULTIPLY complexities
• Dividing/tripling inner loop → logarithmic O(log n)
• Recursion = O(n) space (stack frames) for most linear recursions
• ITERATIVE always wins on SPACE for simple problems

Good luck on Thursday, Thabiso! You've got this!